

PESQUISA – CLEAN CODE

Nome: SOPHIE ALBUQUERQUE DA SILVA

Série: DEVIE



Clean Coode

O que é? - Clean Code (Código Limpo) é uma filosofia de desenvolvimento que prioriza a legibilidade, simplicidade e facilidade de manutenção do código-fonte, garantindo que ele seja facilmente compreendido e modificado por outros desenvolvedores. Popularizado por Robert C. Martin, o conceito foca em evitar complexidade desnecessária e reduzir bugs.

Um código sujo (o oposto do Clean Code) pode funcionar, mas dificulta a evolução do sistema. Gerando alto custo de manutenção a longo prazo.

Resumindo: É uma abordagem que permite às equipes entregar sistemas operantes, intuitivos e facilmente escaláveis.

Robert Cecil Martin percebeu que **o gargalo principal no desenvolvimento de software estava justamente na manutenção**. Ou seja, um código mal escrito desde a sua primeira versão pode funcionar mas vai gerar prejuízos enormes. O termo se disseminou pela comunidade de uma forma praticamente unânime. E um dos motivos pode ser justificado no seguinte dado: **a proporção média de leitura e escrita de códigos fonte é de 10 para 1**. Isso significa que as pessoas passam mais tempo tentando entender os códigos existentes do que efetivamente escrevendo novos códigos. Com essa informação, é possível perceber que, muito além de uma boa arquitetura e boas práticas computacionais, o código fonte – que é aquilo que faz o computador executar nossos comandos – precisa de princípios. E é justamente isso que o Clean Code traz.

-----//-----
Como usado na linguagem Python - Clean Code em Python foca na legibilidade, simplicidade e manutenção do código, seguindo o guia de estilo [PEP 8](#). Práticas essenciais incluem nomes significativos para variáveis e funções, funções curtas com responsabilidade única, uso de constantes, e a escrita de código "pythônico", utilizando estruturas como *list comprehensions* e ferramentas de formatação como Preto.

PEP 8: Seguir o guia de estilo oficial para formatação, como espaços, quebras de linha e nomenclatura.

Nomenclatura Clara: Variáveis e funções devem ter nomes descritivos que revelem sua intenção.

Código "Pythônico": Aproveitar recursos nativos da linguagem para tornar o código mais elegante e simples, evitando soluções complexas (ex: usar `enumerate` ao invés de `range(len())`).

Funções Pequenas: Funções devem realizar apenas uma tarefa.

Uso de Constantes: Definir valores fixos em maiúsculas (ex: `PI = 3.14`) para evitar "números mágicos".

Ferramentas de Linting: Utilizar ferramentas automáticas para garantir a consistência do código, como `black`, `flake8` e `pylint`.

Sete práticas avançadas: Práticas avançadas incluem o uso de type hinting, tratamento de exceções específico e a criação de código modular.

Clean Code em Python significa:

- código fácil de ler
- funções pequenas
- nomes claros
- sem repetição
- bem organizado



-
- **Como aplicar Clean Code** - Aplicar Clean Code (Código Limpo) envolve escrever código focado em legibilidade, simplicidade e manutenção. As principais práticas incluem usar nomes significativos para variáveis e funções, criar funções pequenas que realizam uma única tarefa, evitar comentários desnecessários, seguir o princípio DRY (Don't Repeat Yourself) e garantir tratamento de erros.

Nomes Significativos e Claros: Escolha nomes que contem uma história e revelem intenção. Evite letras soltas (x,y,a) ou abreviações obscuras.

- *Ruim:* var d = 10;
- *Bom:* var diasParaVencimento = 10;

Funções Pequenas e Únicas (SRP): Funções devem ser pequenas e fazer apenas uma coisa bem feita. Se uma função faz muitas coisas, quebre-a em funções menores

.Evite Comentários Desnecessários: Se o código precisa de comentários, ele provavelmente não é claro o suficiente. Use nomes de variáveis e funções explicativos em vez de documentar o óbvio.

Princípio DRY (Don't Repeat Yourself): Evite duplicação de código. Crie funções ou componentes reutilizáveis para garantir que a lógica exista em apenas um lugar.

Tratamento de Erros: Trate erros de forma organizada, garantindo que o código continue funcional mesmo em exceções.

Regra do Escoteiro: Deixe o código mais limpo do que você o encontrou. Refatore pequenos trechos a cada commit.

Adoção de Clean Code resulta em sistemas mais fáceis de manter, menor tempo para implementar novas funcionalidades e maior facilidade para trabalhar em equipe.



Onde usar o Clean Code - O Clean Code (Código Limpo) deve ser aplicado em todas as etapas do desenvolvimento de software, desde a concepção até a manutenção, visando criar códigos legíveis, testáveis e fáceis de manter. É ideal para equipes que buscam aumentar a produtividade e reduzir erros em projetos.

Nomenclatura (Variáveis, Funções, Classes): Use nomes significativos e pronunciáveis, evitando abreviações obscuras ou "data", "info"

.Funções e Métodos: Mantenha-os pequenos, com uma única responsabilidade (SRP), evitando métodos gigantes.

Estruturas Condicionais (if, else, switch): Reduza a complexidade, evitando aninhamentos exagerados. Prefira if simples e estruturas claras.

Comentários: Utilize apenas quando necessário. O código deve ser autoexplicativo, evitando comentários óbvios.

Tratamento de Erros: Crie exceções claras e tratamentos contextualizados, separando o tratamento de erros da lógica principal.

Testes Automatizados: Escreva testes limpos, legíveis e rápidos, pois eles garantem a segurança na refatoração.

Configurações: Mantenha dados de configuração fora do código fonte, de preferência em arquivos separados

Iniciantes/Júnior: A partir do momento em que já entende a sintaxe da linguagem e quer evoluir profissionalmente.

Manutenção: Ao modificar códigos legados, aplicando a refatoração para torná-los melhores, como sugerido pela SiDi

Desenvolvimento de novas funcionalidades: Aplicar desde o início para evitar acúmulo de dívida técnica

```
2 const name = 'John'
3 const lastName = 'Doe'
4 function getExchangeRate(amount, costCurrency){
5   const total = amount * costCurrency
6   return(total)
7 }
8
9 function getWelcomeText(name, lastName){
10  const fullName= name + ' ' + lastName
11  const welcomeText= 'Dear ' + fullName + ','
12  return (welcomeText)
13 }
14
15
16
17
18
```

❌

```
2 const name = 'John'
3 const lastName = 'Doe'
4 function getExchangeRate(amount, costCurrency){
5   const total = amount * costCurrency
6   return(total)
7 }
8
9
10
11
12 function getWelcomeText(name, lastName){
13  const fullName= name + ' ' + lastName
14  const welcomeText= 'Dear ' + fullName + ','
15  return (welcomeText)
16 }
17
18
```

✅



